

Ant-Game 实验要求

实验目标

本实验要求你在 Ant-Game（蚁群陷役）环境下实现一个可运行的 AI。Ant-Game 是一个双人回合制策略游戏，双方在六边形网格地图上对抗：每回合自动在己方基地生成工蚁，工蚁沿路径进攻敌方基地；同时可以消耗金币建造、升级或降级防御塔进行防守，也可使用四种超级武器（闪电风暴、EMP 轰炸、引力护盾、紧急回避）改变局部战局。

你的 AI 将作为一方玩家，根据当前回合、己方座位与游戏状态，输出合法操作序列；系统会依次执行这些操作并更新状态，直至回合结束。

通过本实验，你将掌握：

- 理解游戏规则与状态结构（六边形地图、工蚁、防御塔、超级武器、基地、金币等）；
- 实现符合规范接口的 AI 类，并能正确构造各类操作参数；
- 在本地与评测环境中运行、调试并评估你的 AI。

仓库与代码结构

本仓库提供一个完整的双人回合制策略游戏开发与评测框架，包含以下模块：

- SDK/ – 游戏后端规则、训练环境、动作候选生成与工具
 - SDK/backend/ – 游戏状态、规则推进、回合结算
 - SDK/training/ – 训练环境、自对弈、策略基类
 - SDK/utils/ – 常量、几何、动作、特征提取
- AI/ – 选手 AI（示例：ai_example.py、ai_greedy.py、ai_random.py、ai_mcts.py）
- game/ – C++ 评测器（与 Python SDK 规则一致）
- tools/ – 本地对战、打包脚本等

主目录保留 README.md 作为游戏规则与使用说明的入口。

任务说明

1. 实现 AI 入口

在仓库的 AI/ 目录下新增或修改 AI 模块，文件命名为 ai_<你的名字或标识>.py（例如 ai_greedy.py、ai_zhangsan.py）。该模块必须对外提供 AI 类，继承 BaseAgent，并实现：

```
def choose_bundle(  
    self,  
    state: BackendState,  
    player: int,  
    bundles: list[ActionBundle] | None = None,  
) -> ActionBundle:  
    ...
```

返回值为一个 ActionBundle，其 operations 为当回合要执行的操作列表（建造、升级、降级塔、超级武器、基地升级等），详见「规范」文档。

2. 合法性与运行环境

你发出的每条操作都会由游戏逻辑（BackendState.can_apply_operation）进行合法性检查；非法操作会被拒绝，不会修改状态。

AI 会在 tools/run_local_match.py 的本地对战中按回合被调用；也会在 Saiblo 等评测平台上通过 AI/main.py 的协议入口被使用。

请阅读「通过例子学习」与「规范」文档，确保操作格式、参数范围与游戏规则一致。

3. 可选扩展

- 可使用仓库提供的训练脚本（SDK/train_example.py、SDK/train_mcts.py）进行强化学习或策略梯度训练；
- 可使用 SDK.utils.actions.ActionCatalog 生成候选动作，并在其上实现打分、搜索等策略；
- 权重与辅助代码可放在 AI/<your_name>/ 下，只需保证主入口 AI/ai_<your_name>.py 导出 AI 类即可。

提交与测试

代码提交

提交包含 AI/ai_<你的标识>.py 的仓库（或指定分支），确保在实验指定的 Python 环境下可被 tools/run_local_match.py 及评测系统正确加载（即 package_ai.sh 打包后能正常运行）。

本地测试

单局对战并生成回放：

```
python tools/run_local_match.py --ai0 greedy --ai1 random --seed 7
```

若你的 AI 名为 `my_ai`，需先在 `AI/package_ai.sh` 中新增对应 case，并在 `tools/run_local_match.py` 的 `--ai0 / --ai1` 的 `choices` 中加入 `my_ai`，然后执行：

```
python tools/run_local_match.py --ai0 my_ai --ai1 greedy --seed 7
```

当前内置可选 AI：`random`、`mcts`、`greedy`。

训练示例（验证环境）：

```
bash SDK/train_example.sh --seed 1 --max-actions 16
bash SDK/train_mcts.sh --episodes 2 --iterations 24 --max-depth 3 --seed 1
```

自动化测试：

仓库内 `tests/` 下有接口与对战相关测试，运行 `pytest -q` 可进行整体回归。提交前请确保本地 `pytest -q` 通过。

评测方式

以课程/助教通知为准（可能使用 Saiblo 平台或 CI 脚本对指定 AI 进行多局评测）。

参考资源

- 规范：AI 接口、操作编码、BackendState 与观测结构。
- 通过例子学习：从示例 AI 入手理解接口与操作构造、运行与评测流程。
- 仓库根目录 `README.md`：游戏规则详解、SDK 使用方法、运行与评测教程。